Sitios Web

INTRODUCCIÓN

- Vimos como crear servidores con Express.
  - Servidores HTTP para crear una API REST, Servidores Web etc.
- Tambíen vimos como servir contenido estático

INTRODUCCIÓN

- Pero, ¿y si deseamos contenidos dinámicos?
  - Por ejemplo, para páginas que requieran ir actualizándose (por ejemplo la página de noticias de un periódico)

INTRODUCCIÓN

- Para generar contenido dinámico:
  - Node.JS + Express + handlebars
  - Por dinámico entendemos que el contenido que voy a mostrar en mi sitio web puede venir de diferentes fuentes: base de datos, servicio web, ficheros de texto, etc.

Handlebars. Contextualización

- Es un “motor de templating”
  - Es un motor de vistas ➔ Me permite construir dinámicamente páginas en mi site.

Handlebars. Creación de proyecto (I)

- Creamos un proyecto

```
D:\Desarrollo\Node\dinamico>npm init
This utility will walk you through creating a package.json file.
It only covers the most common items, and tries to guess sensible defaults.

See `npm help init` for definitive documentation on these fields
and exactly what they do.

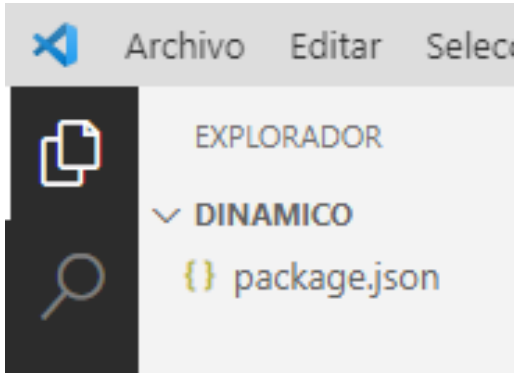
Use `npm install <pkg>` afterwards to install a package and
save it as a dependency in the package.json file.

Press ^C at any time to quit.
package name: (dinamico)
version: (1.0.0)
description: Manjeo de contenido dinámico
entry point: (index.js)
test command:
git repository:
keywords:
author: Pepito Grillo
license: (ISC)
```

```
About to write to D:\Desarrollo\Node\dinamico\package.json:

{
  "name": "dinamico",
  "version": "1.0.0",
  "description": "Manjeo de contenido dinámico",
  "main": "index.js",
  "scripts": {
    "test": "echo \"Error: no test specified\" && exit 1"
  },
  "author": "Pepito Grillo",
  "license": "ISC"
}


Is this OK? (yes) y
```



Handlebars. Creación de proyecto (II)

- Instalamos Express

```
D:\Desarrollo\Node\dinamico>npm i express

added 57 packages, and audited 58 packages in 5s

7 packages are looking for funding
  run `npm fund` for details

found 0 vulnerabilities
```

Handlebars-Express

- Veremos como usar handlebars con Express para crear un sitio web dinámico.

Handlebars-Express (I)

- En primer lugar veamos que es handlebars.js
  - https://handlebarsjs.com/

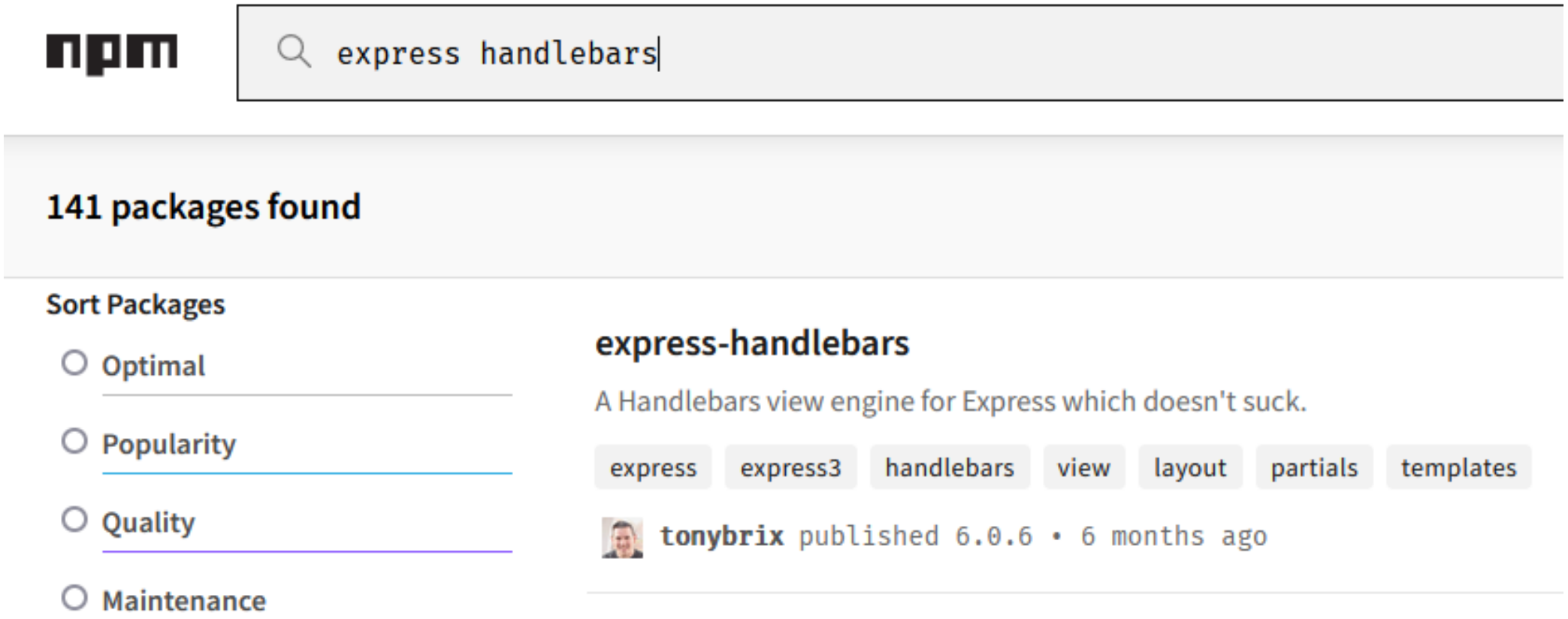
- Veremos lo principal, pero el site es bastante completos con diversos ejemplos.
- Para evitar usar handlebars en crudo y tener que instalarlo y conectarlo con Express, utilizaremos un paquete npm que nos facilite el trabajo.

Handlebars-Express (II)

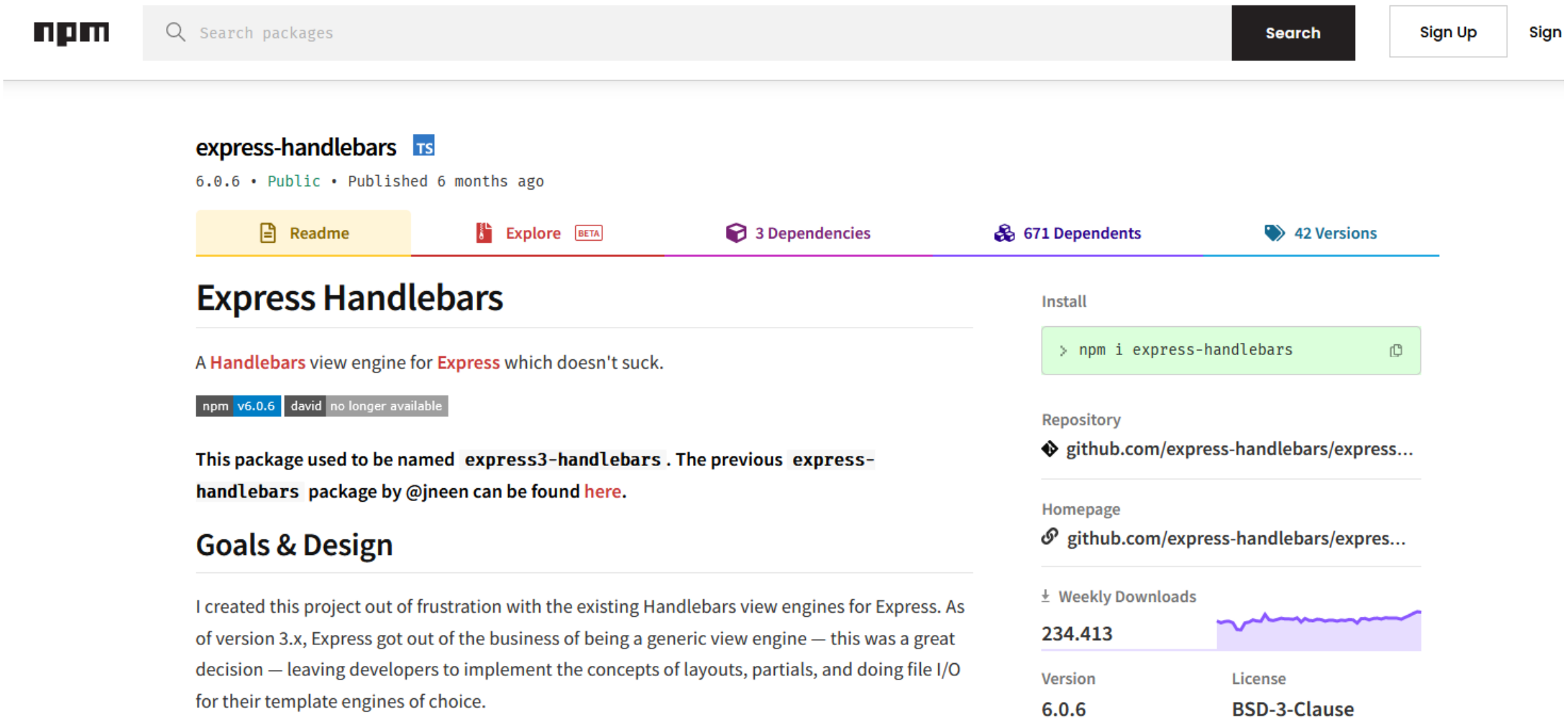
- Para evitar usar handlebars en crudo y tener que instalarlo y conectarlo con Express, utilizaremos un paquete npm que nos facilite el trabajo.

Handlebars-Express (III)

- Utillizaremos express-handlebars



Handlebars-Express (IV)



Handlebars-Express (V)

- Instalamos express-handlebars

```
D:\Desarrollo\Node\dinamico>npm i express-handlebars

added 16 packages, and audited 74 packages in 3s

9 packages are looking for funding
  run `npm fund` for details

found 0 vulnerabilities
```

- Es necesario, para utilizarlo haber instalado antes express
  - De hecho, dado que se pueden instalar varios paquetes de una vez, podíamos haber instalado los dos simultáneamente:
    - npm i express express-handlebars

Handlebars-Express (VI)

- Podemos verificar que ambas dependencias se han incoproprado adecuadamente a nuestro proyecto viendo el package.json:

{} package.json > ...

```
{
  "name": "dinamico",
  "version": "1.0.0",
  "description": "Manjeo de contenido dinámico",
  "main": "index.js",
  ▷ Debug
  "scripts": {
    "test": "echo \"Error: no test specified\" && exit 1"
  },
  "author": "Pepito Grillo",
  "license": "ISC",
  "dependencies": {
    "express": "^4.18.2",
    "express-handlebars": "^6.0.6"
  }
}
```

Handlebars-Express (VII)

- Express handlebars trabaja con “convention over configuration”
  - Express-handlebars espera a que existan ciertas carpetas y ficheros para poder trabajar. Sin necesidad de que hagamos una configuración previa (aunque podríamos).
  - En carpetas de las anteriormente mencionadas se alojarán las vistas y layouts.
    - Un layout se podría descomponer en partes. Por ejemplo: header, footer etc.
    - De esta forma un site con varias páginas que, como sabemos, normalmente comparten estructura general, será mucho más mantenible de esta manera, porque si queremos hacer una modificación en una parte determinada no tendremos que hacerlo en todas y cada una de las páginas, sino en la parte del layout que proceda.

Handlebars-Express (VIII)

- Necesitamos crear la estructura de carpetas necesaria para utilizar handlebars.
  - Una carpeta views
    - Aquí se encontrarán las diferentes vistas (por ejemplo la home, site de clientes, etc.).
    - Dentro de view otra carpeta llamada layouts
      - Aquí tendremos la estructura general del sitio en el que estamos trabajando.
      - Dentro de layout creamos un fichero, que por connevnción usando handlebars llamaremos main.handlebars

Handlebars-Express (IX)

- El contenido de main.handlebars es una web sencilla.
- {{{body}}} ➔ body es una variable interna utilizada para hacer referencia al contenido

views > layouts > main.handlebars > html

```
<!DOCTYPE html>
<html>
<head>
    <meta charset="utf-8">
    <title>Handlebars Template Engine</title>
    <style>
        body {
            background-color: beige;
        }
    </style>
</head>
<body>

    <h1>Utilizando Handlebars!</h1>
   <span>
        <a href="home">Home</a> |
        <a href="clientes">Clientes</a>
    </span>
    <hr>

    <!--INICIO CONTENIDO -->
        {{{body}}}
    <!--FIN CONTENIDO -->

</body>
</html>
```

Handlebars-Express (X)

- De esta forma vamos a tener en nuestras páginas la estructura anteriormente definida y únicamente va a cambiar el body.
  - De hecho lo hemos indicado con {{{body}}} para que handlebar se de cuenta que no es HTML y que ha de cambiarlo por el contenido que le están solicitando

Handlebars-Express (XI)

- Dentro de views creamos otro fichero .handlebars. Yo le voy a llamar home.handlebars pero podría tener otro nombre.
  - Solo ponemos lo que para nosotros sería el contenido de nuestro página home, ya que la estructura común ya ha quedado definida anteriormente.

```
<h2>Home</h2>
<p>
    En un lugar de la Mancha, de cuyo nombre no quiero acordarme, no ha mucho tiempo que
</p>

<div>
    <a href="clientes.html">Clientes</a>
</div>
```

Handlebars-Express (XII)

- A continuación, vamos a montar el servidor. Para ello creamos el fichero index.js dentro de la carpeta de nuestro proyecto e incluimos el código pertinente.

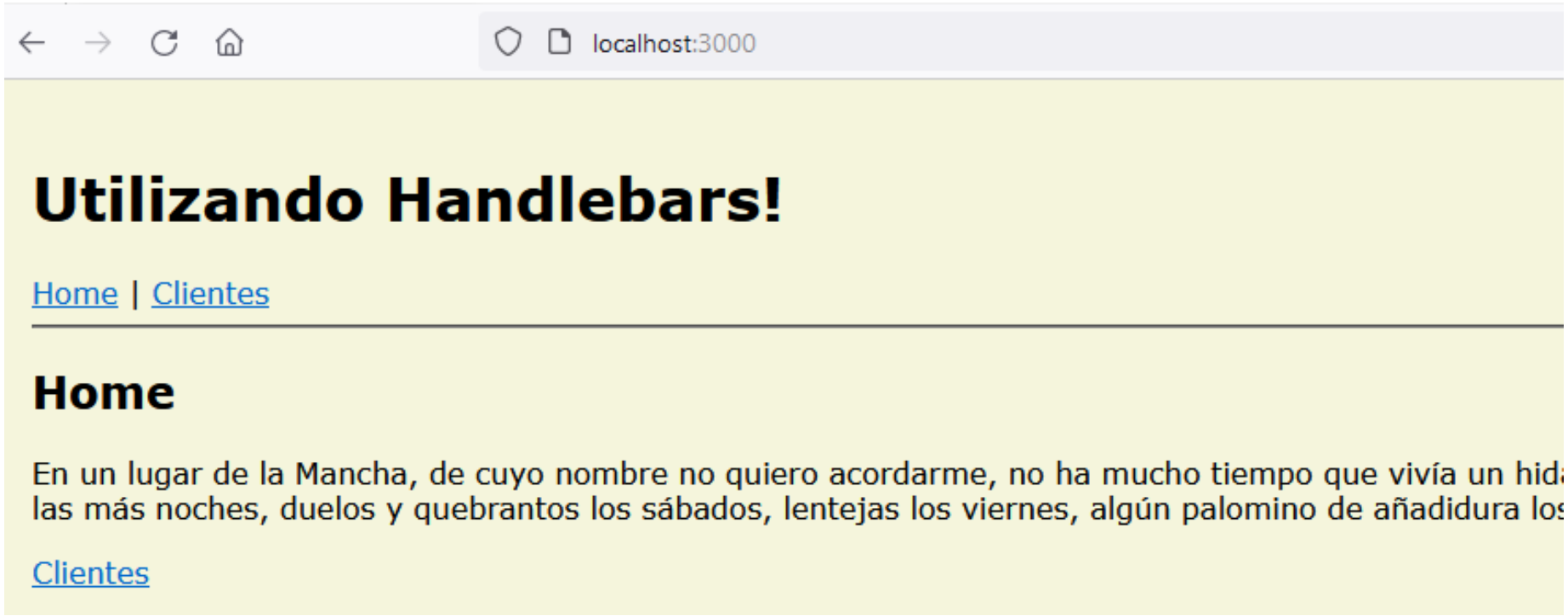
index.js > ...

```
1   const express = require("express");
2   const exphbs = require("express-handlebars");
3
4   // Creamos el servidor de aplicaciones
5   const app = express();
6
7   //Cache
8   //app.enable("view cache");
9
10  //****** Configuracion de Handlebars ******
11  // Informamos Express que vamos a utilizar handlebars de motor de templating
12  // https://expressjs.com/en/5x/api.html#app.engine
13  app.engine("handlebars", exphbs.engine());
14  app.set("view engine", "handlebars");
15
16  // Configuramos el ruteo
17  // En Este caso nos centramos en el raíz
18  app.get("/", (req, res)=>{
19      // res.render ==> Le decmos a Express que no devuelva texto
20      // sino que llame a su motor de vistas
21      // y en este caso a la vista llamada home
22      // home ha de coincidir con el nombre puesto al .handlebars correspondiente
23      res.render("home");
24  });
25
26  app.listen(3000);
```

Handlebars (XIII)

- Probamos lo realizado hasta el momento:

```
D:\Desarrollo\Node\dinamico>nodemon
[nodemon] 2.0.20
[nodemon] to restart at any time, enter `rs`
[nodemon] watching path(s): *.*
[nodemon] watching extensions: js,mjs,json
[nodemon] starting `node index.js`
```



Handlebars-Express (XIV)

- Si inspeccionamos la página, podemos observar que aparece tanto la template como el contenido:

```
<!DOCTYPE html>
<html>
▼ <head>
    <meta charset="utf-8">
    <title>Handlebars Template Engine</title>
  </head>
▼ <body>
    <h1>Utilizando Handlebars!</h1>
  ▼ <span>
      <a href="/">Home</a>
      |
      <a href="clientes">Clientes</a>
    </span>
    <hr>
    <!--INICIO CONTENIDO-->
    <h2>Home</h2>
  ▼ <p>
      En un lugar de la Mancha, de cuyo nombre no quiero acordarme, no ha mucho tiempo que vivía un h
      más vaca que carnero, salpicón las más noches, duelos y quebrantos los sábados, lentejas los vi
    </p>
  ▼ <div>
      <a href="clientes.html">Clientes</a>
    </div>
    <!--FIN CONTENIDO-->
  </body>
</html>
```

Handlebars-Express (XV)

- Ahora montamos la página para clientes. Para ello creamos un fichero llamado clientes.handlebars dentro de views.

```
1  <h2>Clientes</h2>
2
3  <ul>
4      <li>Cliente1</li>
5      <li>Cliente2</li>
6      <li>Cliente3</li>
7  </ul>
8
9  <p>Pues sepa Vuestra Merced, ante todas cosas, que a mí llaman Lázaro de
```

- En el home hacemos una pequeña modificación y en el href en vez de poner la referencia a clientes.html la ponemos a clientes.

```
<h2>Home</h2>
<p>
    En un lugar de la Mancha, de cuyo nomb
</p>

<div>
    <a href="clientes">Clientes</a>
</div>
```

Handlebars-Express (XVI)

- En el index habrá que generar la ruta que atienda al recurso correspondiente:

index.js > ...

```
1   const express = require("express");
2   const exphbs = require("express-handlebars");
3
4   // Creamos el servidor de aplicaciones
5   const app = express();
6
7   //Cache
8   //app.enable("view cache");
9
10  //****** Configuracion de Handlebars ******
11  // Informamos Express que vamos a utilizar handlebars de motor de templating
12  // https://expressjs.com/en/5x/api.html#app.engine
13  app.engine("handlebars", exphbs.engine());
14  app.set("view engine", "handlebars");
15
16  // Configuramos el ruteo
17  // En Este caso nos centramos en el raíz
18  app.get("/", (req, res)=>{
19      // res.render ==> Le decmos a Express que no devuelva texto
20      // sino que llame a su motor de vistas
21      // y en este caso a la vista llamada home
22      // home ha de coincidir con el nombre puesto al .handlebars correspondiente
23      res.render("home");
24  });
25
26  app.get("/clientes", (req, res)=>{
27      res.render("clientes");
28  });
29
30  app.listen(3000);
```

Handlebars-Express (XVII)

Utilizando Handlebars!

Home | Clientes

Home

En un lugar de la Mancha, de cuyo nombre no quiero acordarme, no ha mucho tiempo que vivía un hida viernes, algún palomino de añadidura los domingos, consumían las tres partes de su hacienda

Clientes

- Y si nos vamos a clientes:

Utilizando Handlebars!

Home | Clientes

Clientes

- Cliente1
- Cliente2
- Cliente3

Pues sepa Vuestra Merced, ante todas cosas, que a mí llaman Lázaro de Tormes, hijo de Tomé González y de Antona Pérez, naturales de Tejares, aldea de Salamanca. I perdone, tenía cargo de proveer una molienda de una aceña que está ribera de aquel río, en la cual fue molinero más de quince años; y, estando mi madre una noche en

Handlebars-Express (XVIII)

- Cualquier cosa que cambiemos en el template general nos cambia en todo. Por ejemplo si queremos cambiar el fondo del body:

views > layouts > main.handlebars > html > head > style

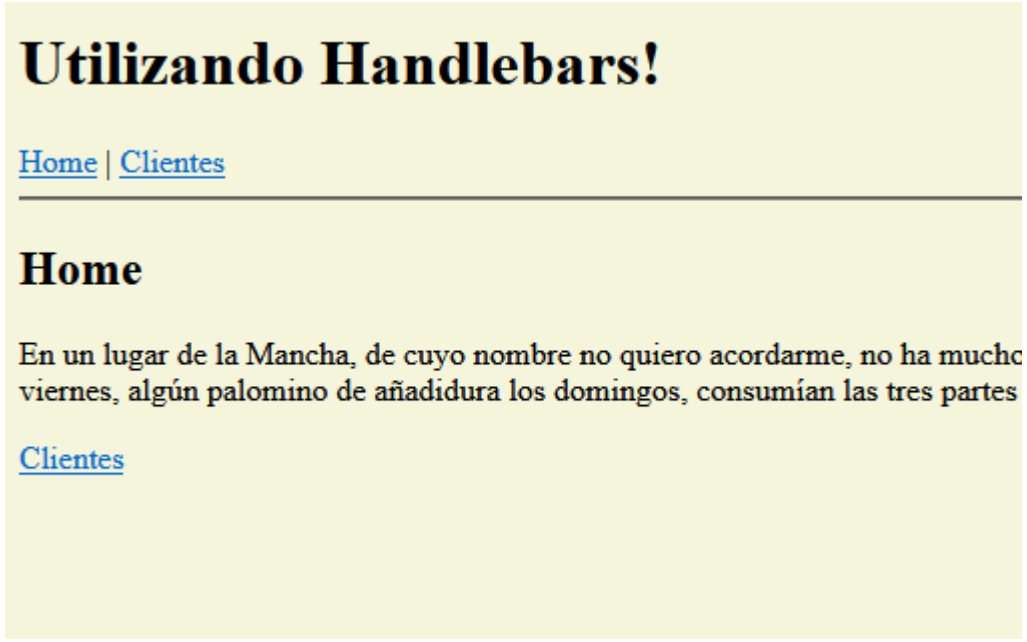
```
<!DOCTYPE html>
<html>
<head>
    <meta charset="utf-8">
    <title>Handlebars Template Engine</title>
    <style>
        body {
            background-color: beige;
        }
    </style>
</head>
<body>

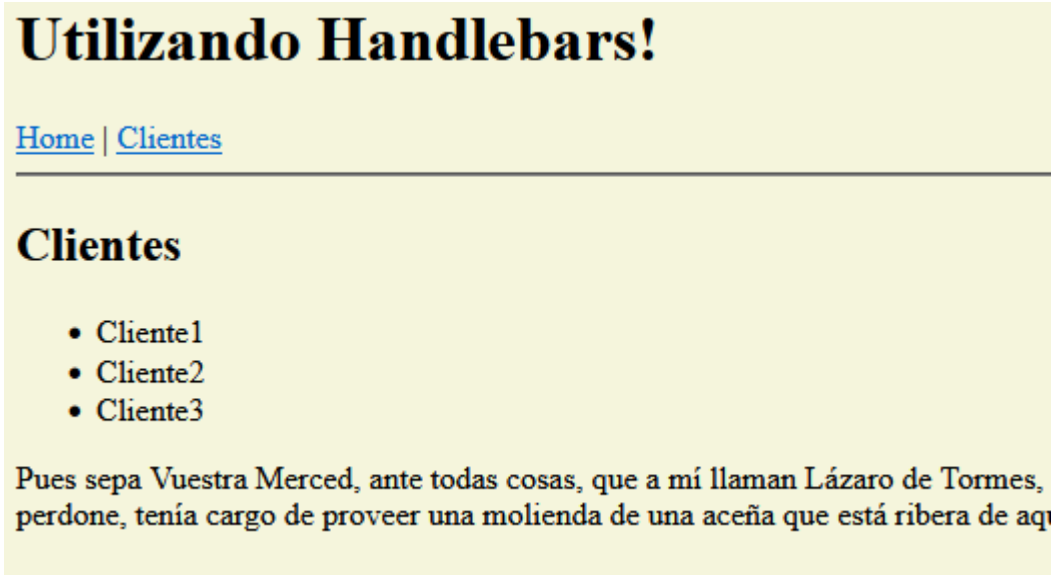
    <h1>Utilizando Handlebars!</h1>
    <span>
        <a href="/">Home</a> |
        <a href="clientes">Clientes</a>
    </span>
    <hr>

    <!--INICIO CONTENIDO -->
        {{{body}}}
    <!--FIN CONTENIDO -->

</body>
</html>
```

Handlebars-Express (XIX)





Handlebars. Cache (I)

- Es importante incluir una política de caché. Que lo que hará será guardar en memoria información para no tener que ir a generarla una y otra vez.
- Es habitual en los web sites guardar cada cierto tiempo la información en caché.
- Al pasar nuestro de desarrollo a producción es interesante activar caché.
- Express nos facilita todo esto.
  - Debemos hacerlo antes de configurar el motor.
  - Como veis en el código estaba comentado, ya que estamos en desarrollo.
  - Si lo descomentásemos, habilitamos caché, pero aun no lo haremos

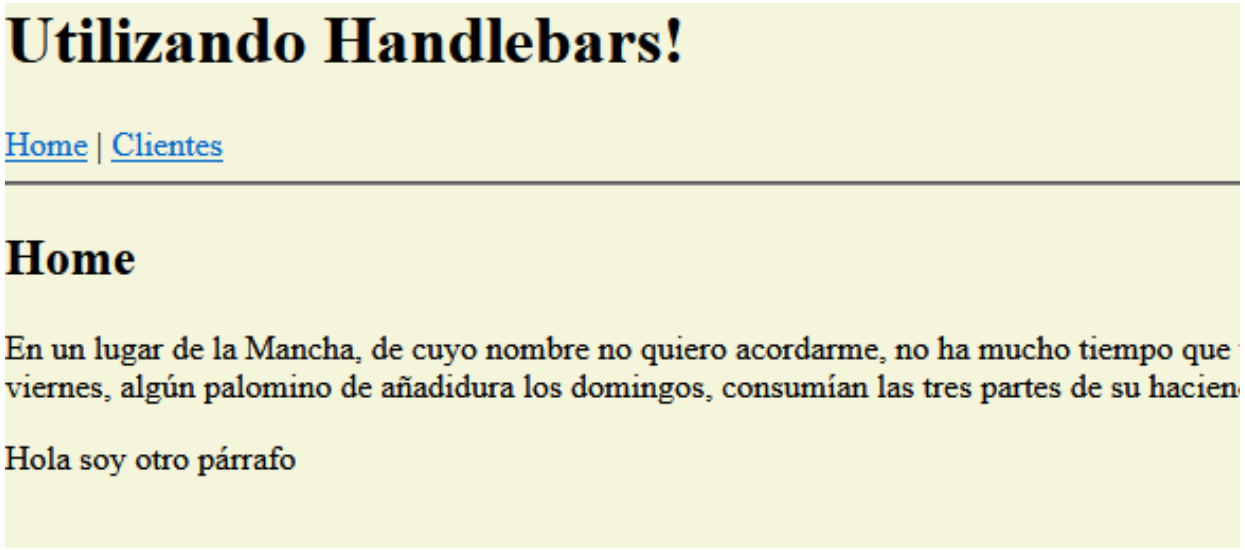
Handlebars. Cache (II)

- Le añadimos otro párrafo al home.
- Además quitamos el div de clientes ya que tenemos el menú para eso.

```
1   <h2>Home</h2>
2   <p>
3       En un lugar de la Mancha, de cuyo n
4   </p>
5
6   <p>
7       Hola soy otro párrafo
8   </p>
9
```

Handlebars. Cache (III)

- Vemos el resultado:

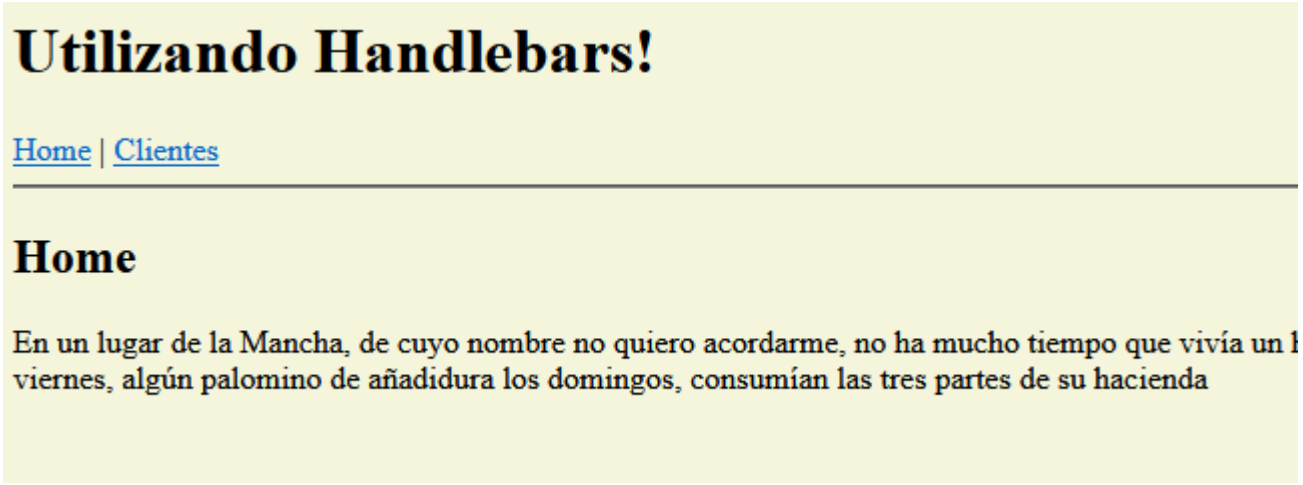


Handlebars. Cache (IV)

- Ahora:
- Activamos caché. Descomentamos la parte del código que teníamos preparada.

```
//Cache
app.enable("view cache");
```

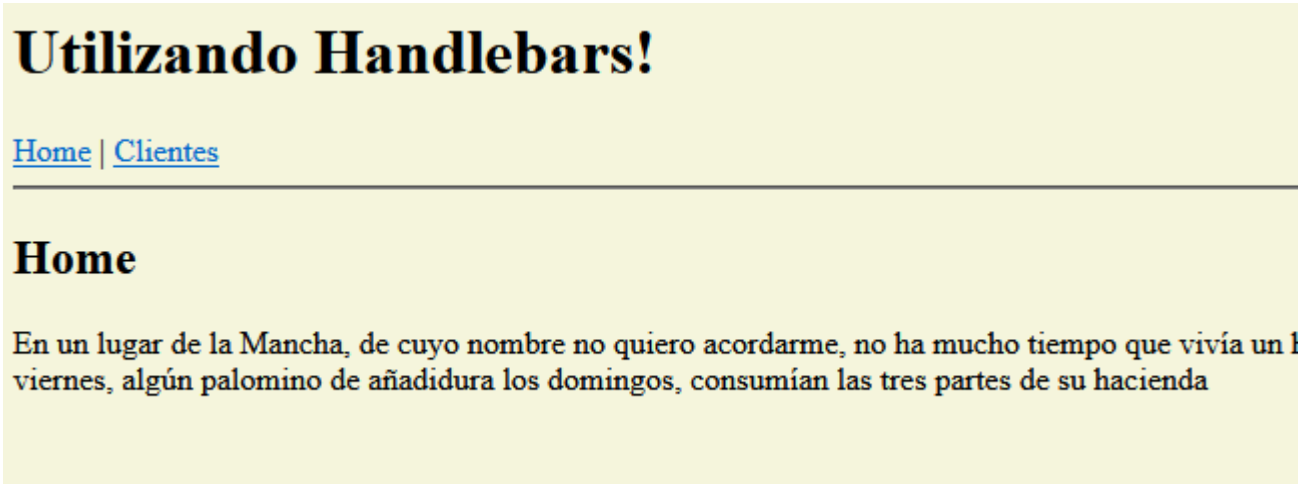
- Comentamos el párrafo nuevo que habíamos incluido y vemos el resultado:



Handlebars. Cache (V)

Lo volvemos a descomentar y probamos:

```
//Cache
app.enable("view cache");
```



Como podemos observar, debido a la información que se tiene en caché, los cambios no se actualizan y sigue sin verse el párrafo, a pesar de que ya no está comentado.

Handlebars. Cache (VI)

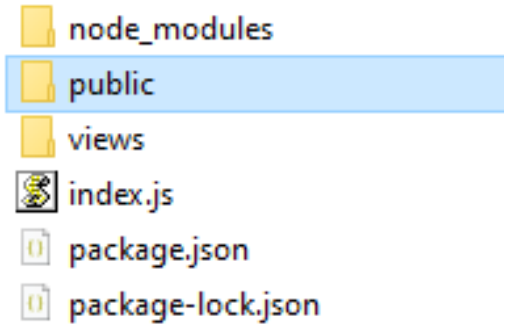
-- Esta política de caché es muy sencilla, porque es “binaria” o hay o no hay, pero puede ser más compleja, por ejemplo en e sentido de actualizar contenidos cada cierto tiempo, o disparar actualización por medio de un flag de cambios que haya en la BDD, etc..

-- Pero, recordemos que utilizar caché es propio de producción y ahora estamos en desarrollo con lo que volvemos comentar esas líneas de código.

```
//Cache
//app.enable("view cache");
```

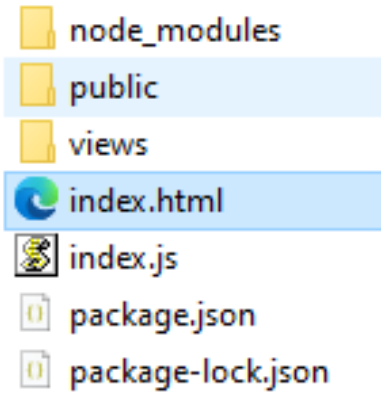
Handlebars. Trabajando con Templates (I)

- Vamos a mejorar un poco el diseño de nuestra página.
- Para ello busquemos un template (de cualquier sitio de templates, por ejemplo en starbootstrap.com).
- Es importante, cuando trabajamos con templates organizar bien todas las partes HTML, CSS, librerías etc
  - La parte del CSS y las librerías incluido en sus carpetas correspondientes y ambas a su vez incluidas en un carpeta public nos la llevamos a la carpeta de nuestro proyecto.



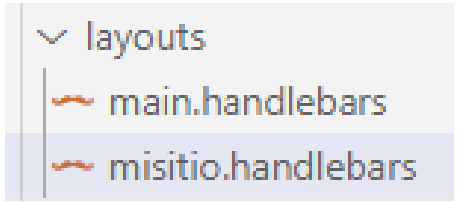
Handlebars. Trabajando con Templates (II)

- El .html nos lo llevamos a la carpeta de nuestro proyecto.
  - No lo vamos a utilizar pero es para poder verlo cómodamente desde el editor.



Handlebars. Trabajando con Templates (III)

- De ese index.html tendríamos que separar el contenido del resto de la estructura de la página.
- En layouts, nos vamos a crear uno nuevo, que llamaremos misitio.handlebars



- Dentro de dicho layout, copiamos y pegamos todo el código del index.html

Handlebars. Trabajando con Templates (IV)

- Pero, recordemos que en los layouts no nos interesa tener el contenido.
- Por ello, localizamos, dentro de misitio.handlebars el contenido. En este tipo de plantillas, suele estar bien indicado. En el caso de la que estoy utilizando se marca claramente con el comentario Page content. No obstante, somos capaces de diferenciarlo, ya que sería la parte que no incluye, head, footer etc.
- Una vez localizado el contenido lo cortamos del fichero misitio.handlebars e indicamos con {{{body}}} que allí se encontraría dicho contenido.

Handlebars. Trabajando con Templates (V)

```
<!-- Header -->
<header class="bg-primary py-5 mb-5">
  <div class="container h-100">
    <div class="row h-100 align-items-center">
      <div class="col-lg-12">
        <h1 class="display-4 text-white mt-5 mb-2">Business Name or Tagline</h1>
        <p class="lead mb-5 text-white-50">Lorem ipsum dolor sit amet, consectetur adipis
      </div>
    </div>
  </div>
</header>

{{{body}}}

<!-- Footer -->
<footer class="py-5 bg-dark">
  <div class="container">
    <p class="m-0 text-center text-white">Copyright &copy; Your Website 2019</p>
  </div>
  <!-- /.container -->
</footer>

<!-- Bootstrap core JavaScript -->
<script src="vendor/jquery/jquery.min.js"></script>
<script src="vendor/bootstrap/js/bootstrap.bundle.min.js"></script>

</body>
```

Handlebars. Trabajando con Templates (VI)

- Generamos el view del layout (le llamaremos misitioHome.handlebars).
- Pegaremos en el view creado el contenido cortado del layout.
- Hacemos en view algunos cambios para que a primera vista veamos mejor los resultados. Sobre todo pasar palabras las castellano:
  - What We Do ➔ Home
  - Conctact ➔ Contacto
  - Call to action ➔ Acción
  - name@example.com ➔ pepitogrillo@otrogmail.com

Handlebars. Trabajando con Templates (VII)

- Ahora tenemos que implementar el ruteo.
- Crearemos una ruta para el nuevo site. Así, creamos la carpeta routes dentro de la carpeta de nuestro proyecto y dentro un fichero que llamaremos misitio.js

misitio.js > ...

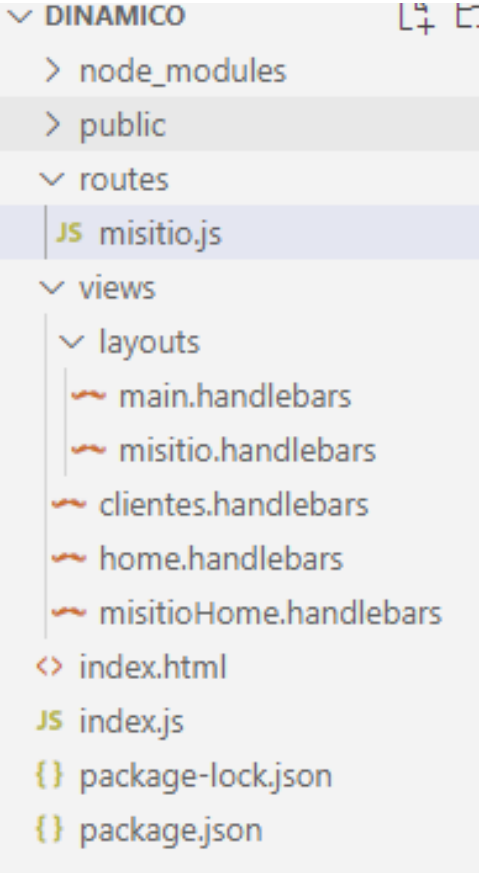
```
const express = require("express");
// Nos traemos específicamente el routeador de express
const router = express.Router();

router.get("/", (req, res) => {

    res.render("misitioHome");

});

module.exports = router;
```



Handlebars. Trabajando con Templates (VIII)

- En el index tendremos que hacer el require de la nueva ruta para poder utilizarla:

```
app.get("/clientes", (req, res)=>{
    res.render("clientes");
});

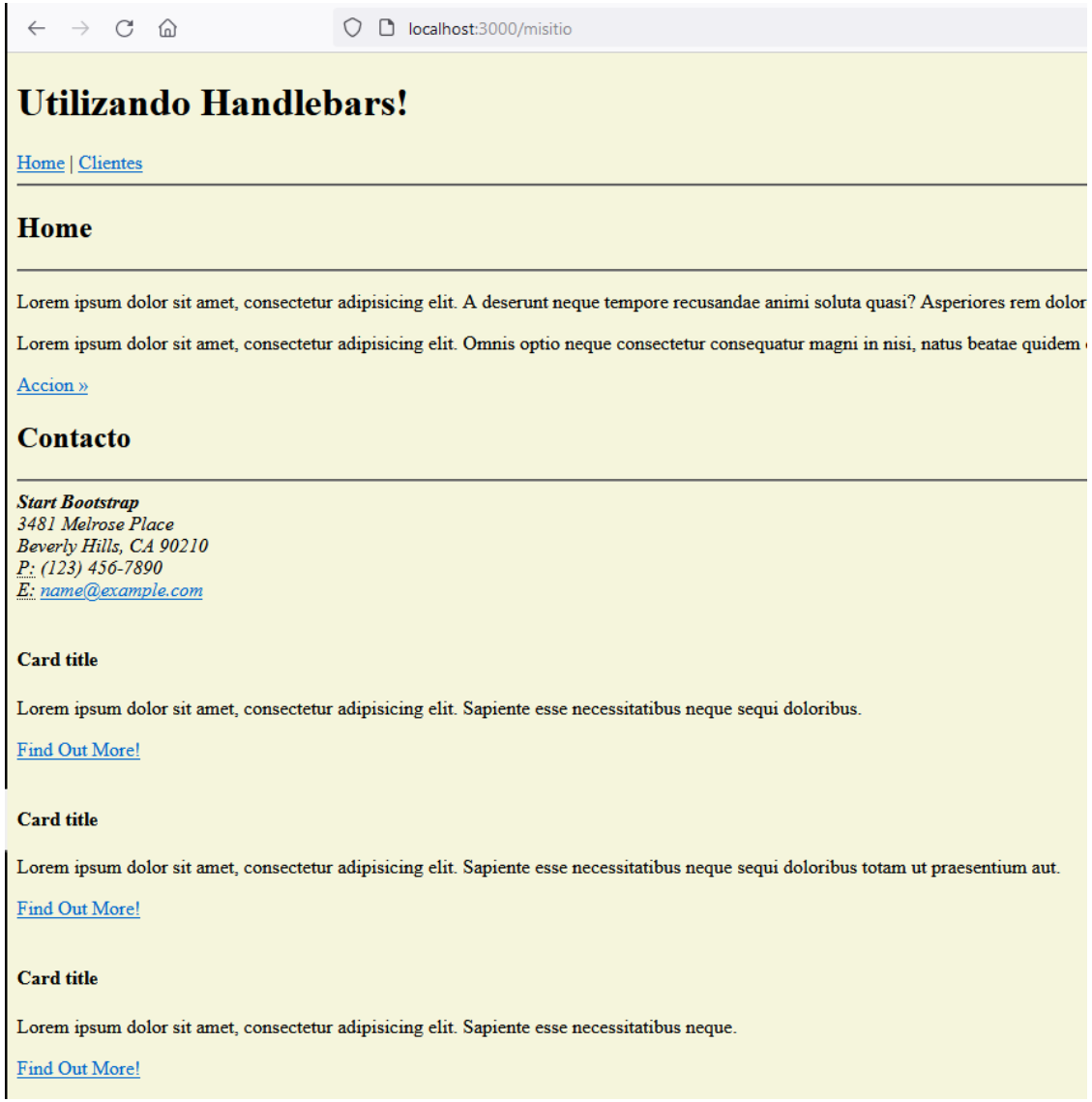
//Nuevo sitio
// Como buena práctica en vez de hacer un ruteo como
// hemos visto hasta ahora
//En este caso construiremos un ruteo nuevo
//para el nuevo site
const mysite = require("./routes/misitio");
app.use("/misitio", mysite);



app.listen(3000);
```

Handlebars. Trabajando con Templates (IX)

- Probamos a ver nuestro nuevo sitio:



Handlebars. Trabajando con Templates (X)

- Como vemos sale una mezcla del layout anterior y la información del template nuevo (del home nuevo).
- Hay que tener en cuenta que nuestro template utiliza su propio estilo y sus propios js. Por ello, tendremos que hacer la configuración correspondiente para que el servidor acceda y pueda servir los estáticos correspondientes:

```
app.get("/clientes", (req, res)=>{
    res.render("clientes");
});

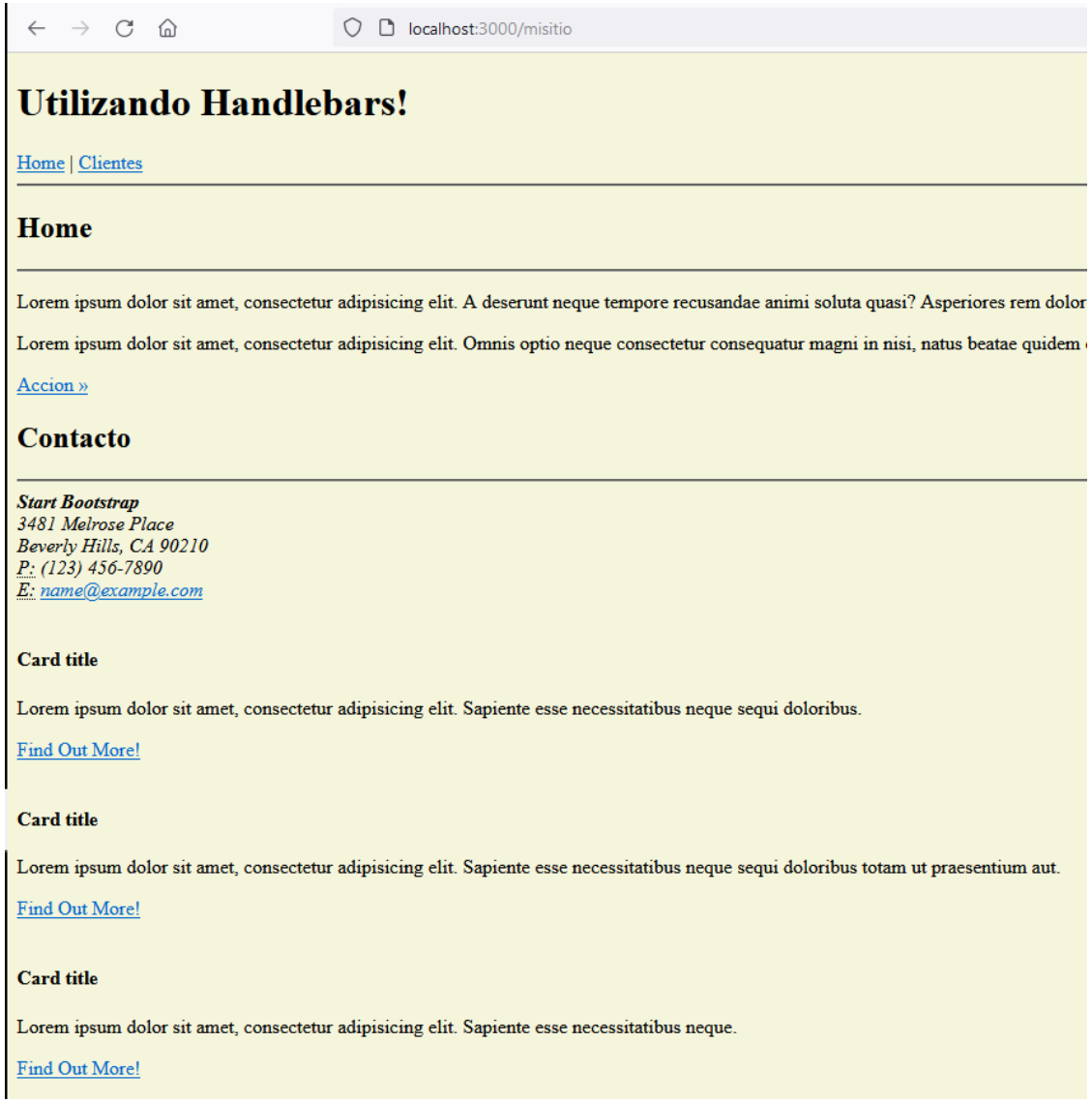

//Nuevo sitio
// Como buena práctica en vez de hacer un ruteo como
// hemos visto hasta ahora
//En este caso construiremos un ruteo nuevo
//para el nuevo site
const mysite = require("./routes/misitio");
app.use("/misitio", mysite);

//Estáticos del sitio nuevo
app.use(express.static("public"));

app.listen(3000);
```

Handlebars. Trabajando con Templates (XI)

- Si probamos a ver nuestro nuevo sitio, vemos que no ha pasado nada:



Handlebars. Trabajando con Templates (XII)

- Esto se debe a que, por defecto, si no le especificamos el layout se va al main que es el único que conoce por ahora express.
- Por tanto, tendremos que decirle que si entra por /misitio vaya al layout mistio.handlebars.
  - De esta manera, podremos utilizar un layout diferente que no sea el main (el default)

```
const express = require("express");
// Nos traemos específicamente el routeador de express
const router = express.Router();

router.get("/", (req, res) => {

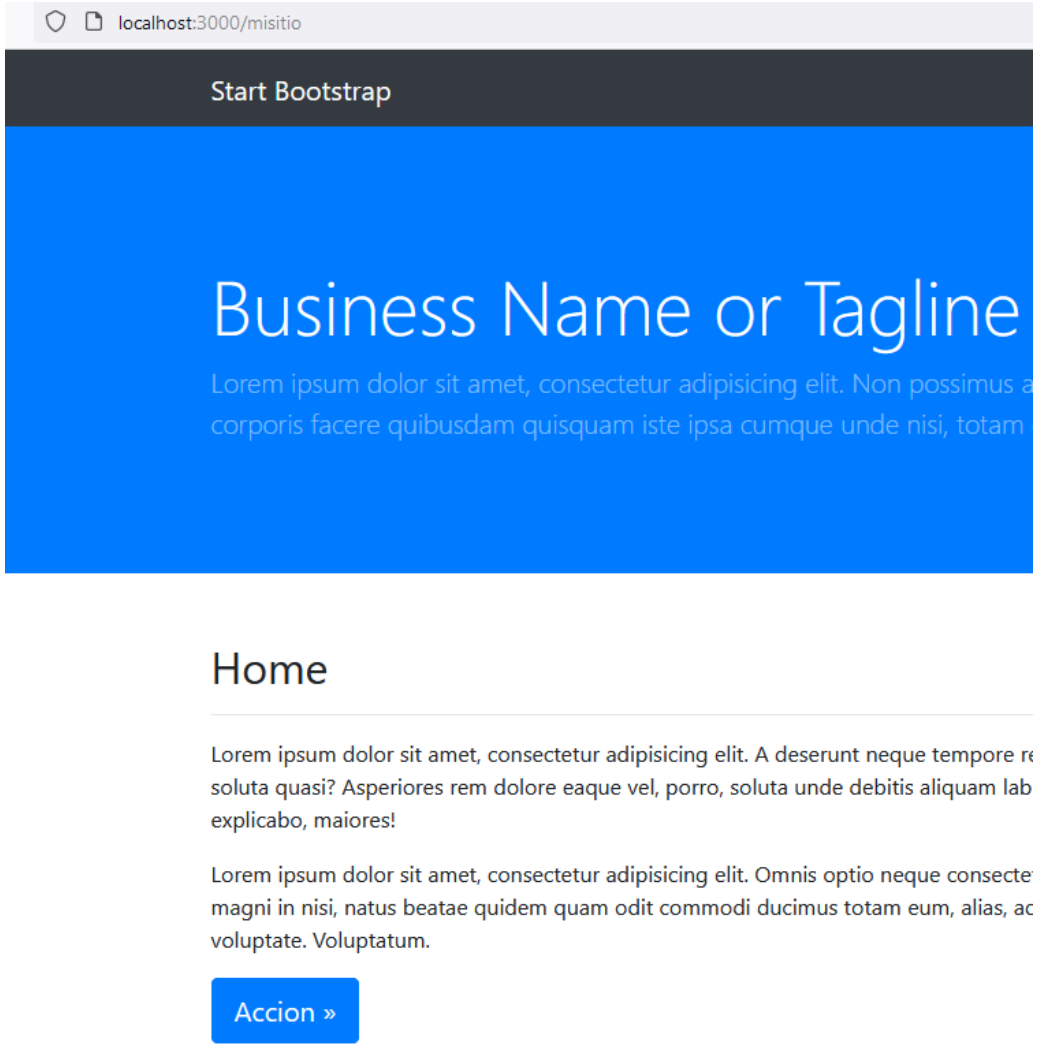
    // pasamos un objeto con la propiedad layout indicando
    // indicando el nombre del layout correspondiente
    res.render("misitioHome", {layout:"misitio"});

});

module.exports = router;
```

Handlebars. Trabajando con Templates (XIII)

- Si probamos a ver nuestro nuevo sitio:



Handlebars. Trabajando con Templates (XIV)

- Todavía se le puede sacar más partido a handlebars, por ejemplo, busándo contenido público en internet, transformándolo y haciendo que se muestre dentro de mi site, utilizándolo, por tanto en mis propias vistas.

Handlebars. Trabajando con Templates (XV)

- Añadimos a en nuestro layout main algunos aspectos en el style.

layouts > main.handlebars > html > head > style > body

```
<!DOCTYPE html>
<html>
<head>
    <meta charset="utf-8">
    <title>Handlebars Template Engine</title>
    <style>
        body {
            background-color: beige;
            font-family: "Verdana";
            padding:1em;
        }
    </style>
</head>
<body>

    <h1>Utilizando Handlebars!</h1>
   <span>
        <a href="/">Home</a> |
        <a href="clientes">Clientes</a>
    </span>
    <hr>

    <!--INICIO CONTENIDO -->
        {{{body}}}
    <!--FIN CONTENIDO -->

</body>
</html>
```



Handlebars. Trabajando con Templates (XVI)

- Veamos como le podemos pasar información al template que provenga de una BDD, de internet, etc.
- En este caso, lo que haremos será pasar un objeto JSON al render

```
// Configuramos el ruteo
// En Este caso nos centramos en el raíz
app.get("/", (req, res)=>{
    // res.render ==> Le decmos a Express que no devuelva texto
    // sino que llame a su motor de vistas
    // y en este caso a la vista llamada home
    // home ha de coincidir con el nombre puesto al .handlebars correspondiente
    // podríamos pasar al render un objeto JSON y
    // mediante clave valor pasar información
        res.render("home", {
            title: "Bienvenidos a mi sitio!",
            subtitle: "Gracias por venir!",
            arrLang: ["Javascript", "C#", "PHP", "Java", "Python"],
            content: "Bla Bla Bla etc...."
        });
    });

app.get("/clientes", (req, res)=>{
    res.render("clientes");
});
```

Handlebars. Trabajando con Templates (XVII)

- Nos vamos a home handlebars y podemos poner algo que venga de fuera utilizando los placeholders de handlebars. De hecho, le diremos que cojacontenido que le llegue a través de Express y, concretamente, los campos que hemos metido en el objeto JSON anterior.
  - En interesante observar que podemos utilizar condicionales y bucles.
    - Si viene la propiedad arrLAng creo la lista. (utilizon un if).
    - Dentro del ul utilizo un bucle para generar cada li.

home.handlebars > ...

```
<h2>{{title}}</h2>
<h3>{{subtitle}}</h3>
<p>
    {{content}}
</p>

{{#if arrLang}}
<ul>
    {{#each arrLang}}
        <li>{{this}}</li>
    {{/each}}
</ul>
{{/if}}
```

Tarea: Investigar aparte del if y el bucle mostrado todos los recursos de programación que nos ofrece handlebars.

Handlebars. Trabajando con Templates (XVIII)

- Veamos el resultado:



Esto hace a handlebars muy potente: puedo traer información de BDD, de internet etc. y construir con ella un layout y vista específica

Handlebars. Ejemplo Api I

- Ahora, utilizando el nuevo sitio haremos una petición a una API externa y utilizaremos la información recibida para generar contenido en nuestro layout.
- Instalaremos request, recordad que está deprecada y tenéis como ejercicio utilizar otra. Por tanto, podréis aplicarlo posteriormente a este ejemplo.

```
D:\Desarrollo\Node\dinamico>npm i request
npm WARN deprecated har-validator@5.1.5: this library is no longer supported
npm WARN deprecated uuid@3.4.0: Please upgrade  to version 7 or higher.  Older ve
 circumstances, which is known to be problematic.  See https://v8.dev/blog/math-r
npm WARN deprecated request@2.88.2: request has been deprecated, see https://gith

added 43 packages, and audited 117 packages in 6s

10 packages are looking for funding
  run `npm fund` for details

found 0 vulnerabilities
```

Handlebars. Ejemplo Api II

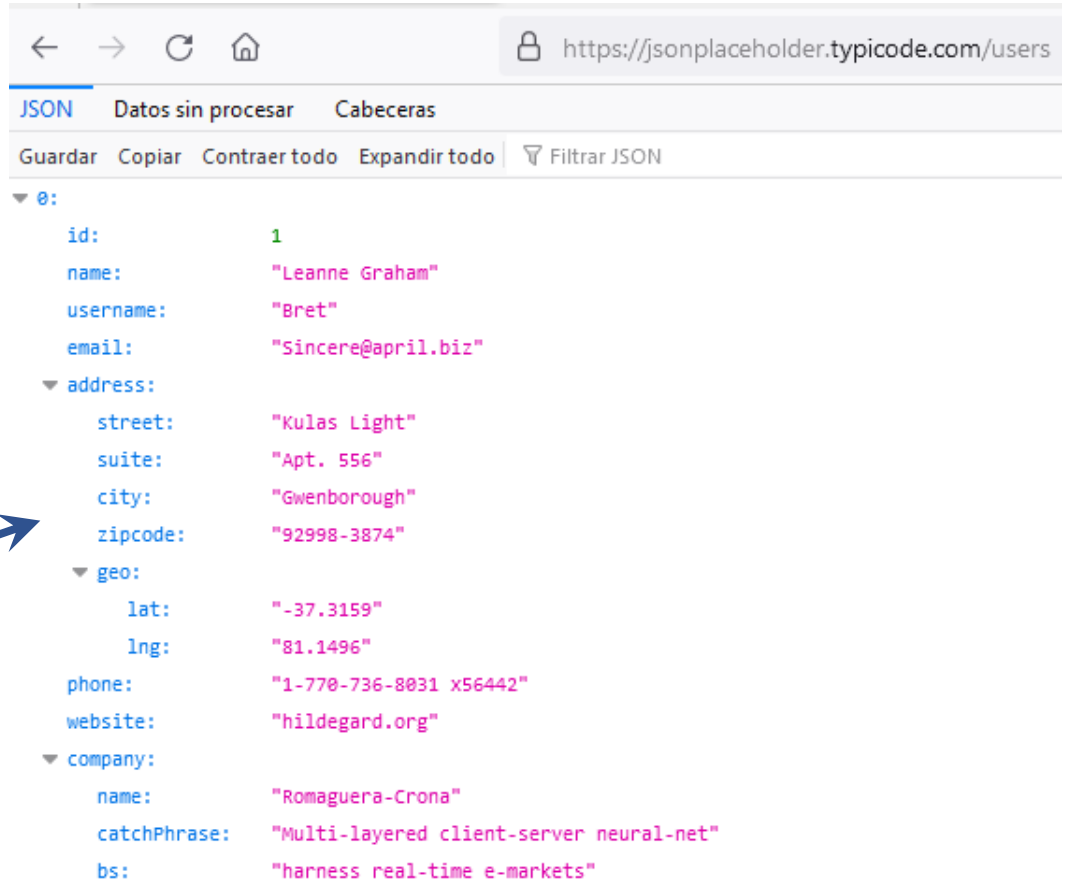
- Utilizaremos el sitio web jsonplaceholder
  - https://jsonplaceholder.typicode.com/

- Son APIs públicas con diferentes datos.
- Si nos fijamos hay varios sets de datos, utilizamos users porque es la más pequeña.

Resources

JSONPlaceholder comes with a set of 6 common resources:

/posts	100 posts
/comments	500 comments
/albums	100 albums
/photos	5000 photos
/todos	200 todos
/users	10 users



Handlebars. Ejemplo Api III

- Hacemos las modificaciones oportunas en misitio.js:

```
const express = require("express");
// Nos traemos específicamente el routeador de express
const router = express.Router();
const request = require("request");

router.get("/", (req, res) => {


  //Obtener listado de usuarios
    request("https://jsonplaceholder.typicode.com/users", (err, response, body)=>{
        if(!err){
            const users = JSON.parse(body);

            res.render("misitioHome", {
                layout:"misitio",
                users : users
            });
        }
    });


});

module.exports = router;
```

Handlebars. Ejemplo Api IV

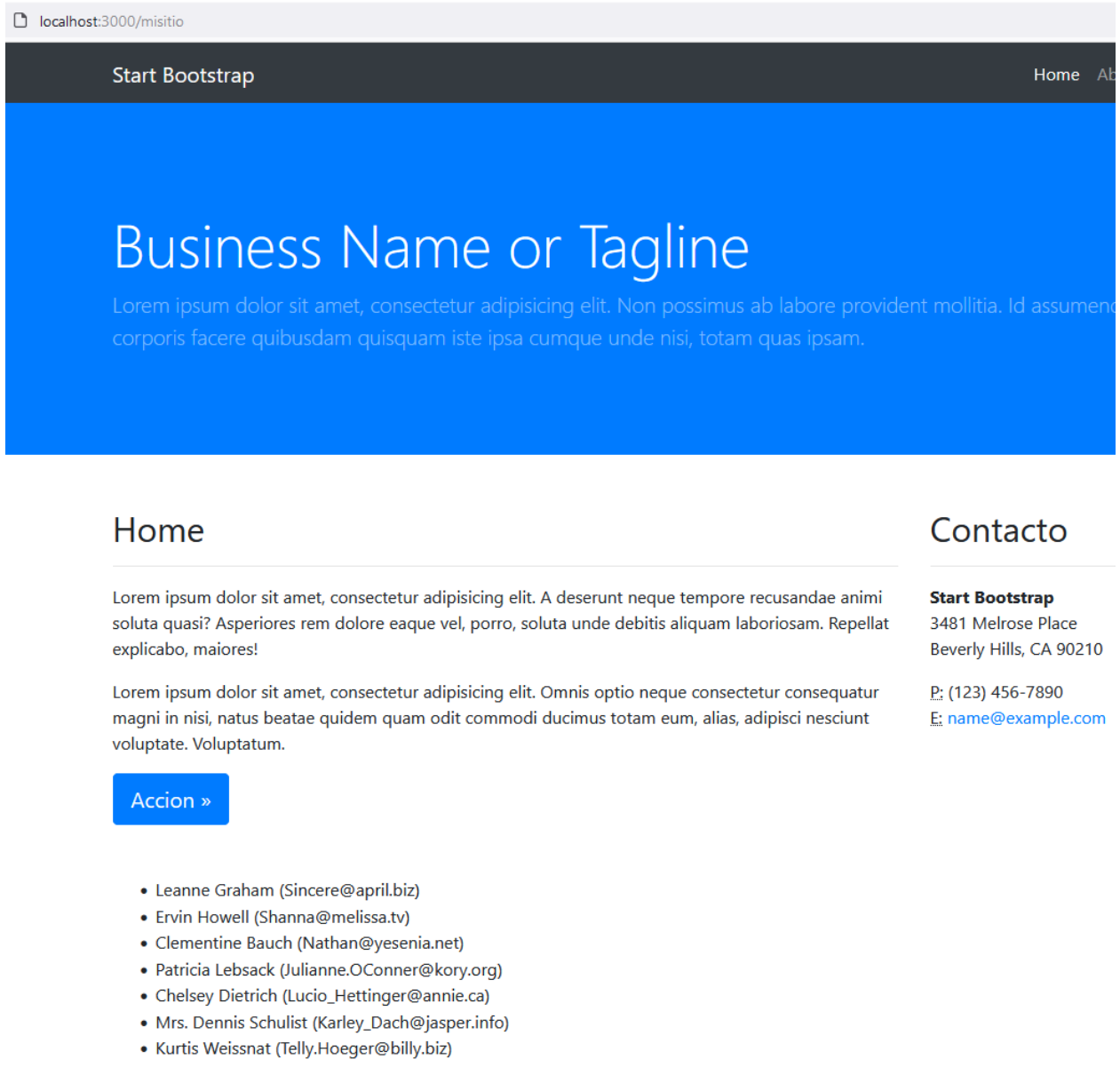
- En misitiohome.handlebars realizamos alguna modificación para mostrar el contenido adecuadamente:

```
        <br>
        <abbr title="Email">E:</abbr>
        <a href="mailto:#">name@example.com</a>
      </address>
    </div>
  </div>
  <!-- /.row -->
  <!-- MYSITE -->
  {{#if users}}
      <ul>
          {{#each users}}
              <li>{{this.name}} ({{this.email}})</li>
          {{/each}}
      </ul>
  {{/if}}

  <div class="row">
    <div class="col-md-4 mb-5">
      <div class="card h-100">
        <img class="card-img-top" src="http://placehold.it/300x200" alt="">
        <div class="card-body">
```

Handlebars. Ejemplo Api V

- Veamos como quedaría y como nos traemos información de la API:



Handlebars. Ejemplo Api VI

- Procesar la request de este tipo en cliente o en servidor es una cuestión de diseño. En este caso, las ventajas de procesarla desde servidor son principalmente:
  - No se puede trackear de donde viene la información.
  - En caso que fuera una request que fuera lenta y tardar en procesase no se le quedaría la ventana “colgada” al usuario.